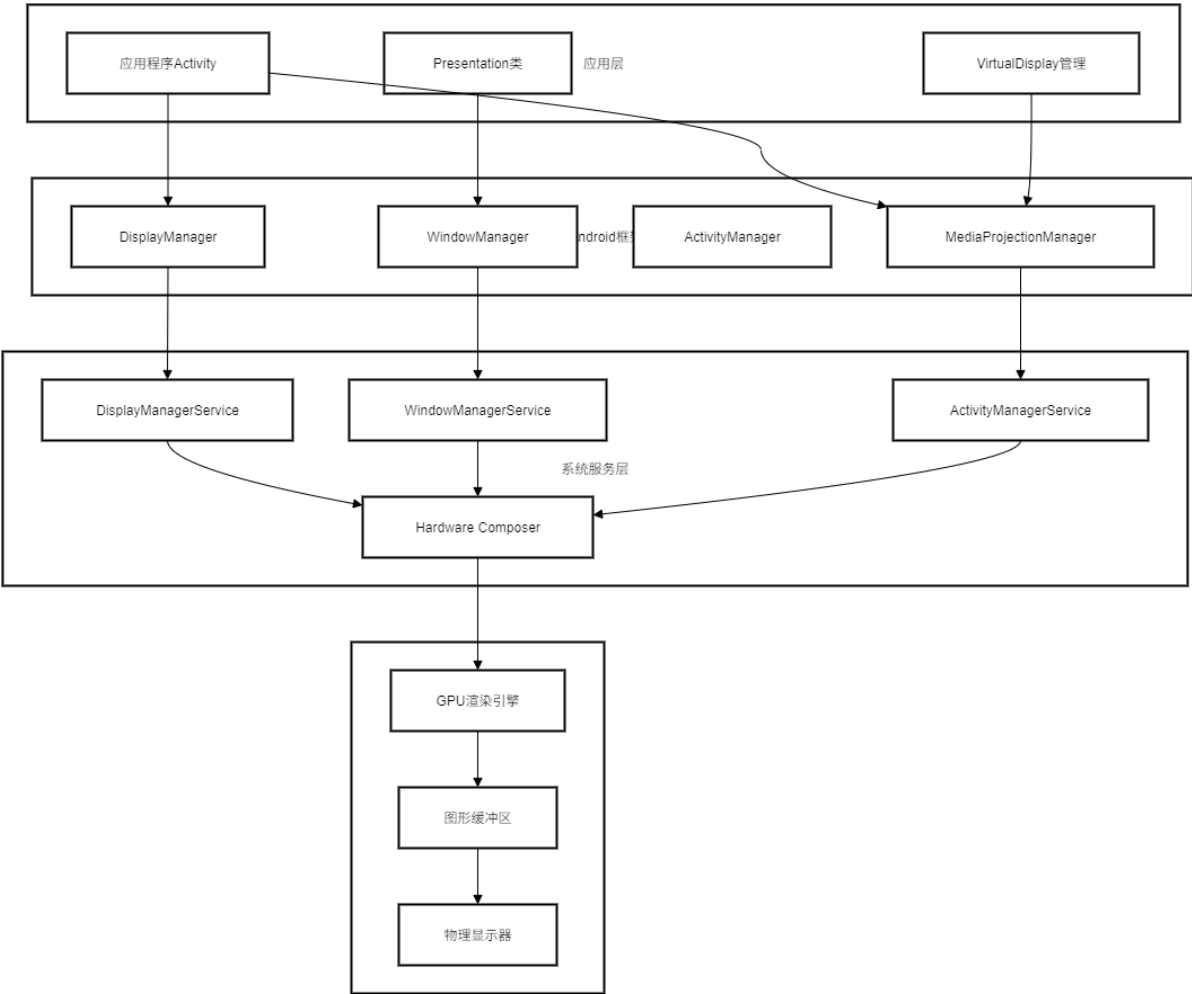


摘要

随着智能座舱的快速演进，多屏化、智能化已成为定义下一代车载体验的核心。副驾驶娱乐屏、后排娱乐屏乃至仪表盘的数字化，对底层的操作系统提出了前所未有的技术挑战。本报告从技术架构师的视角，深入剖析了车载系统中实现副屏显示的两条核心技术路线。首先，报告将详尽分析Android Automotive OS (AAOS) 生态中的原生显示技术，重点阐述 **Presentation API** 和 **VirtualDisplay** 的工作原理、API细节、性能考量及其在车载环境下的固有局限性。随后，报告将以理想L系列等量产车型为案例，剖析其如何在原生Android框架上通过深度定制，构建出高性能、高体验的多屏交互系统。作为对比，报告将引入华为鸿蒙座舱（AITO问界系列）的“分布式软总线”与“原子化服务迁移”架构，揭示其与Android原生方案在设计哲学、实现机制和生态系统上的根本性差异。最终，本报告旨在为车载信息娱乐（IVI）系统架构师、开发者及产品决策者提供一份兼具技术深度与战略洞察的参考，以应对未来座舱多屏HMI设计的挑战与机遇。

第一章：Android原生多屏显示技术基础

Android作为一个通用操作系统，其原生框架提供了管理多显示屏的基础能力。然而，这些API并非专为车载多用户、多任务并发场景设计，理解其工作原理与内在限制，是探究车载座舱技术方案的前提。



1.1 Presentation API：轻量化的副屏内容呈现方案

`Presentation` API 是Android提供的一种在副屏上显示内容的标准方式。然而，从架构层面理解，`Presentation` 并非一个独立的渲染上下文或应用实例，而是 `Dialog` 的一个特化子类。这一本质区别决定了其核心特性与应用局限。

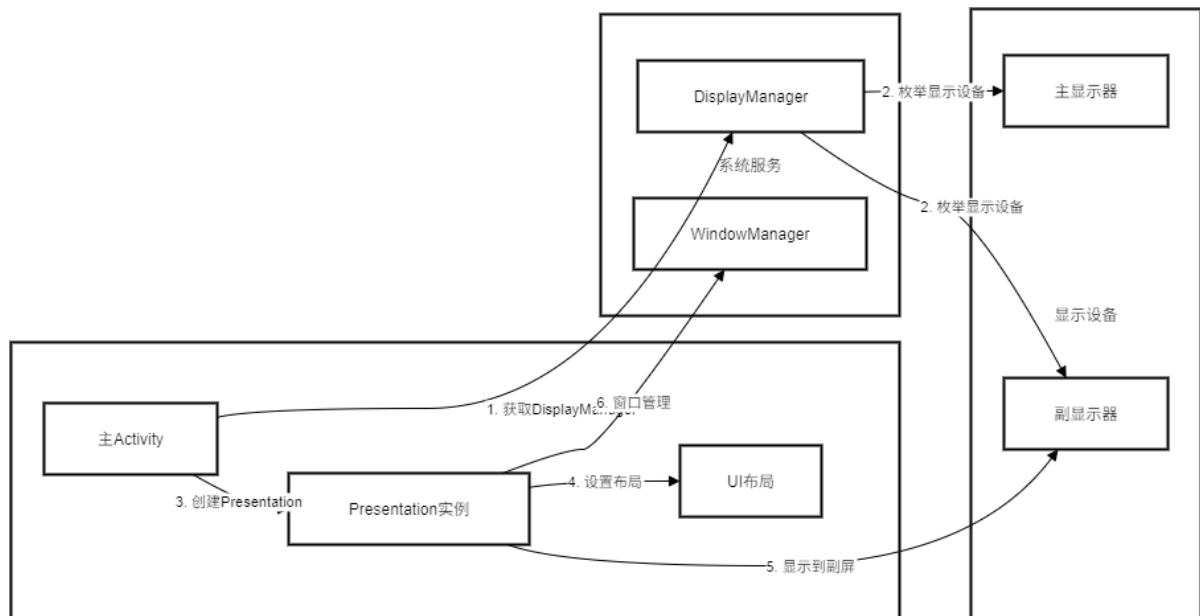
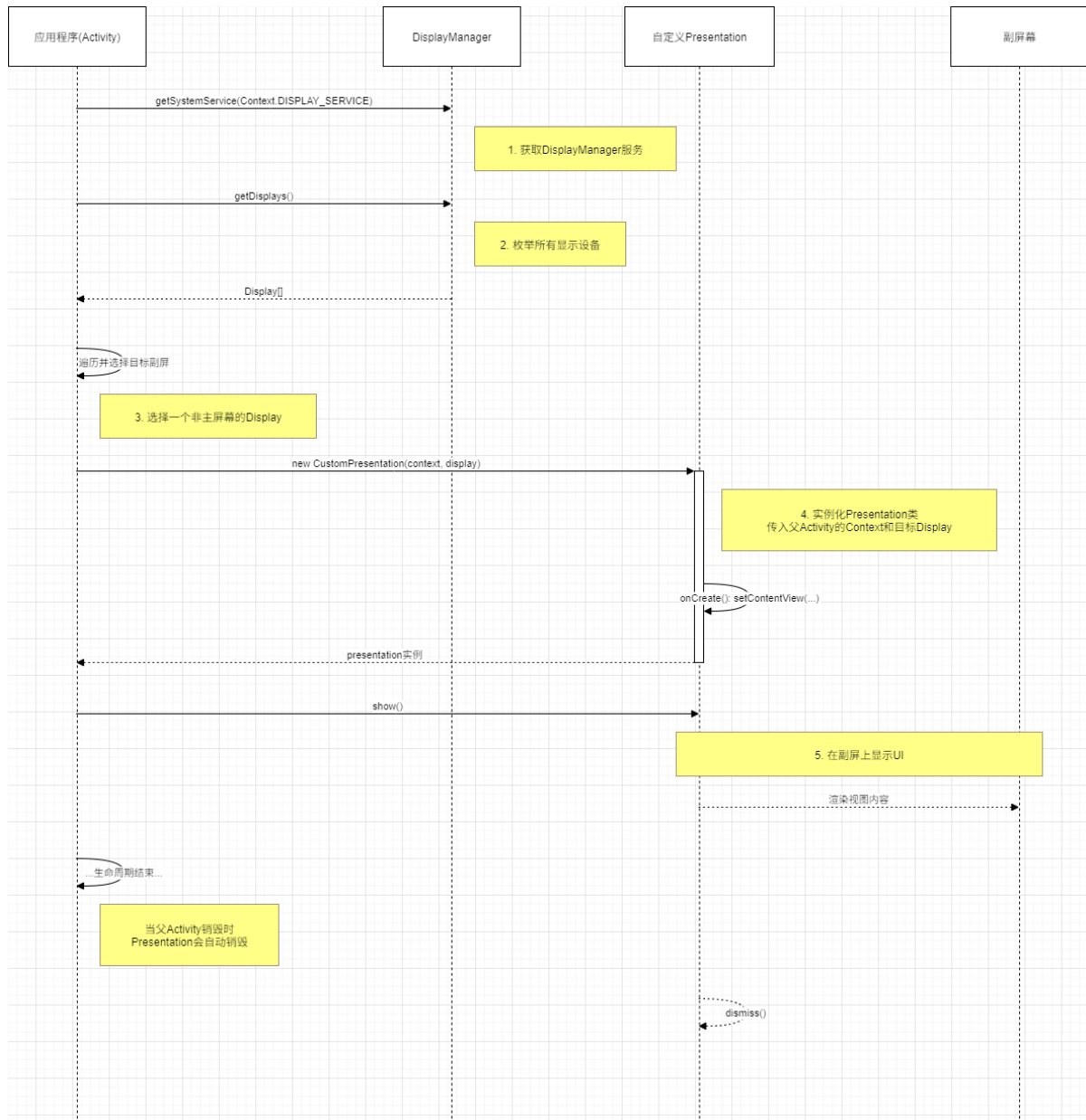
1.1.1 架构原理与实现流程

`Presentation` 实例在生命周期和上下文层面，与创建它的父 `Activity` 紧密耦合。其设计初衷是在代表某个应用向第二屏幕“呈现”内容，而非在副屏上运行一个独立的、可交互的应用程序。

其典型实现流程如下：

1. **获取DisplayManager**：通过 `Context` 获取 `DisplayManager` 系统服务实例。
2. **枚举显示设备**：调用 `getDisplay()` 方法获取当前连接的所有显示设备列表。系统通常将主屏幕定义为 `DEFAULT_DISPLAY`。
3. **选择目标副屏**：遍历显示设备列表，选择一个非主屏幕的 `Display` 对象作为内容呈现的目标。
4. **实例化Presentation**：创建一个继承自 `Presentation` 的自定义类，并在其构造函数中传入父 `Activity` 的 `Context` 和目标 `Display` 对象。在其 `onCreate()` 回调中，加载并设置要显示的UI布局。
5. **显示与销毁**：调用 `show()` 方法将内容呈现在副屏上，并随父 `Activity` 的销毁而自动销毁。

自Android 10起，该API被正式确立为支持车载外部显示（如仪表盘）的方式之一。



1.1.2 车载应用场景与核心局限

`Presentation` 的轻量化特性使其适用于一些简单的车载场景，例如在仪表盘上显示简化的逐向导航箭头、媒体播放的曲目信息或来电通知。然而，对于现代智能座舱所追求的丰富乘客娱乐体验，

`Presentation` 存在三个致命的局限：

- **单一音频焦点 (Single Audio Focus)**：这是最关键的限制。由于 `Presentation` 与父应用共享同一个音频焦点，导致无法实现驾驶员收听导航语音，而副驾乘客通过同一应用观看带声音的电影。一旦副驾屏幕播放视频，就会抢占整个应用的音频焦点，中断驾驶员的导航播报。
- **无法运行独立应用**：该API无法投射一个完整的、独立的 `Activity` 或应用。它只能显示其 `Dialog` 内部的视图内容，功能受限于单个视图层级。
- **交互能力受限**：`Presentation` 默认不直接处理触摸事件。若要实现复杂的交互，如游戏或视频播放控制，需要开发者实现一套独立的、复杂的事件注入流程，开发成本高且体验不佳。

1.2 VirtualDisplay API：灵活的屏幕镜像与渲染框架

与 `Presentation` 相比，`VirtualDisplay` 是一种功能更强大、更底层、也更复杂的机制。它不直接与 UI 组件绑定，而是提供了一个虚拟的、存在于内存中的显示设备，其核心是与 `MediaProjection` API 协同工作，构建一个灵活的内容捕获与渲染管线。

1.2.1 架构原理与实现流程

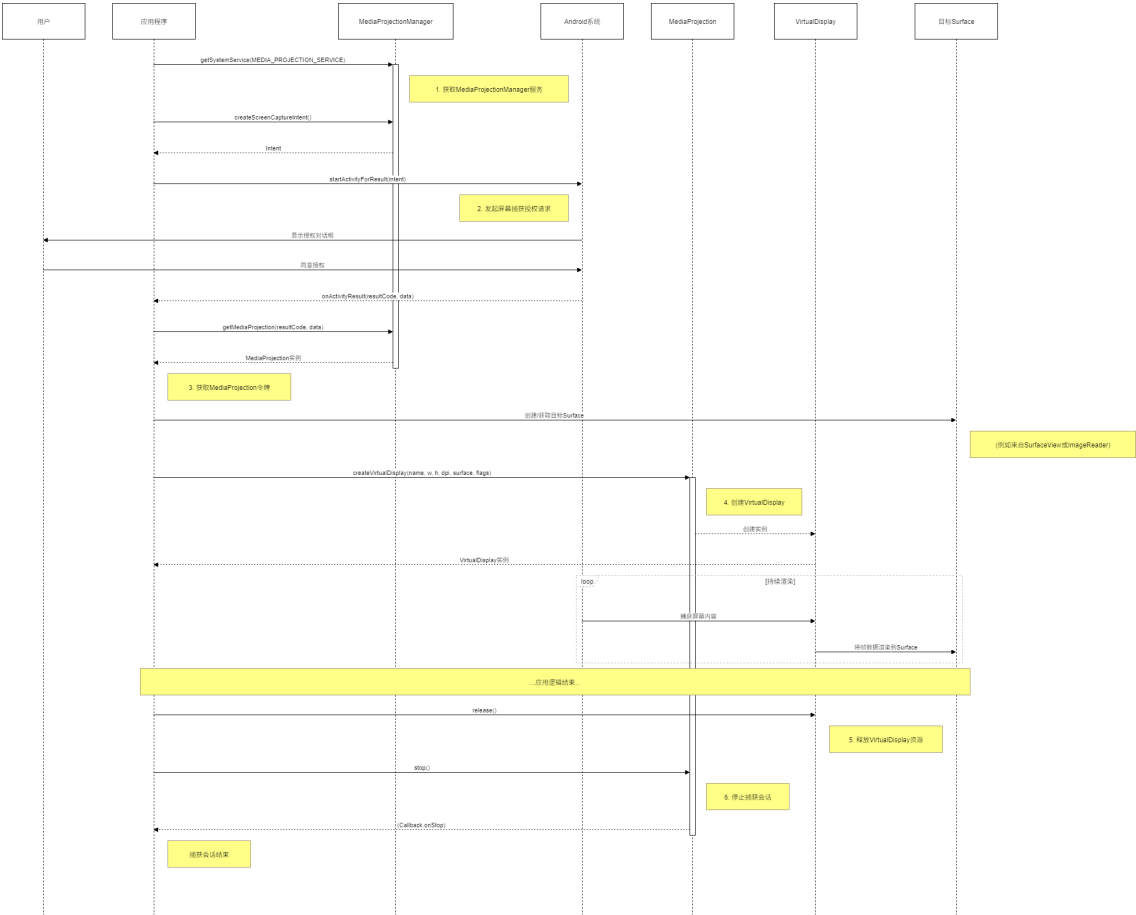
`VirtualDisplay` 的核心思想是创建一个虚拟显示器，系统将指定内容（如整个屏幕或特定应用的窗口）合成并渲染到这个虚拟显示器上，然后将渲染结果的每一帧都输出到一个由应用程序提供的 `Surface` 上。这个

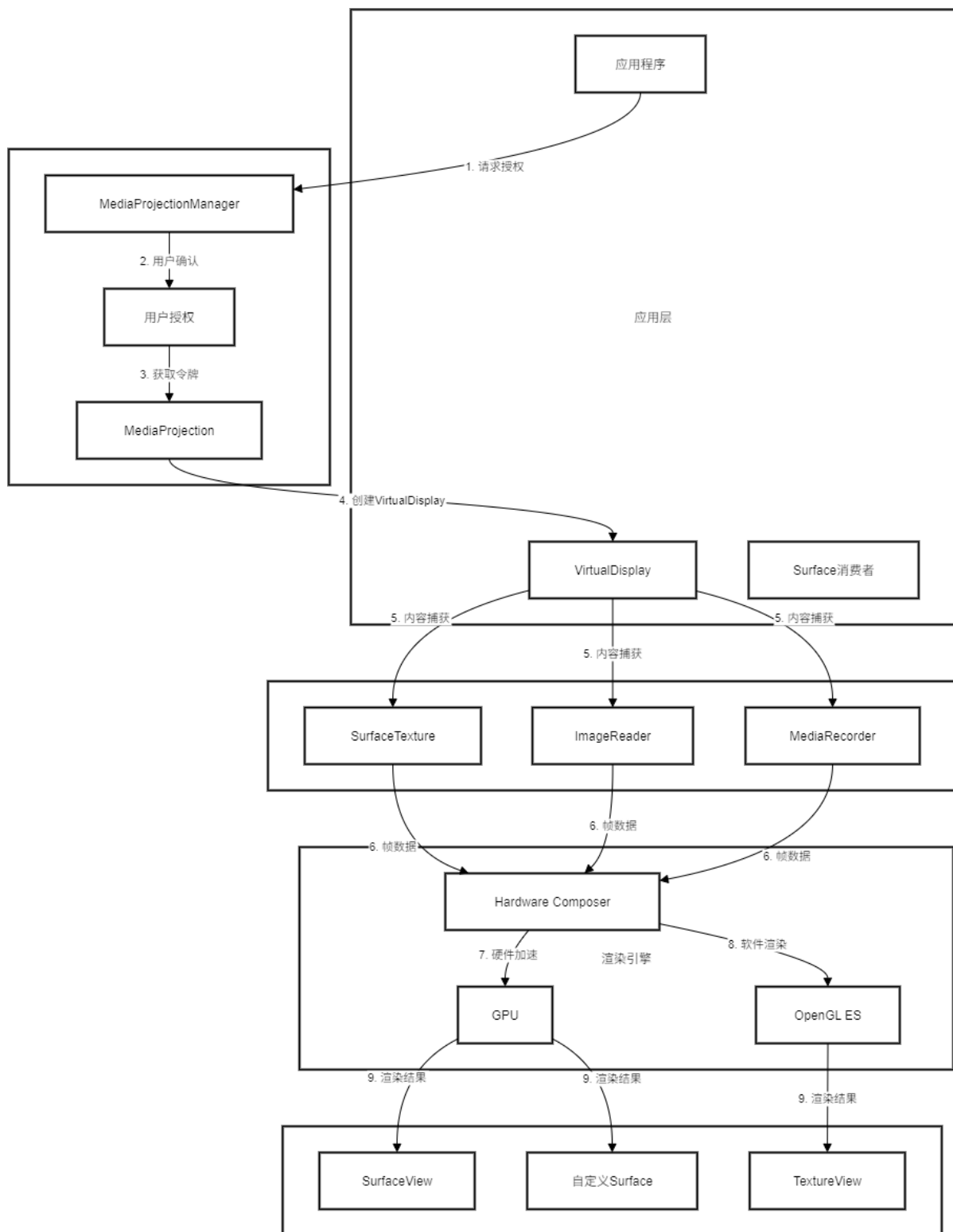
`Surface` 是整个管线的关键连接点，它可以是：

- `MediaRecorder` 的输入 `Surface`，用于屏幕录制。
- `ImageReader` 的 `Surface`，用于逐帧捕获图像并进行实时处理或分析。
- 另一个 UI 组件（如 `SurfaceView` 或 `SurfaceTexture`）的 `Surface`，用于将捕获的内容再次显示在屏幕上，实现“画中画”或投屏效果

其标准实现流程包括：

1. **获取授权**：通过 `MediaProjectionManager` 的 `createScreenCaptureIntent()` 方法发起请求，由用户授权屏幕捕获权限，并在 `onActivityResult` 中获取 `MediaProjection` 令牌实例。
2. **创建VirtualDisplay**：调用 `MediaProjection` 实例的 `createVirtualDisplay()` 方法。此方法需要传入虚拟显示器的名称、尺寸（宽度、高度）、DPI 密度以及一些标志位，如 `DisplayManager.VIRTUAL_DISPLAY_FLAG_AUTO_MIRROR`（自动镜像内容）。
3. **提供目标Surface**：将预先创建好的目标 `Surface` 传入 `createVirtualDisplay()` 方法。从此刻起，系统便开始将内容持续渲染到这个 `Surface` 上。





1.2.2 性能与资源管理

在对性能和资源极其敏感的车载环境中，使用 `VirtualDisplay` 必须审慎处理以下问题：

- 性能调优：** `VirtualDisplay` 本身不提供直接控制帧率的API。在某些场景下，如远程投屏或仪表盘状态更新，过高的帧率会造成不必要的CPU/GPU负载和网络带宽消耗。一种常见的优化手段是引入一个中间 `SurfaceTexture`。`VirtualDisplay` 将内容渲染到此 `SurfaceTexture` 上，应用监听其 `onFrameAvailable` 回调。在回调中，应用可以根据业务逻辑（如定时器、内容变化检测）决定是否将这一帧纹理通过GLES渲染到最终的目标 `Surface` 上，从而实现对帧率的主动控制。

- **内存管理：**`VirtualDisplay` 是一个重量级资源，如果使用不当，极易引发内存泄漏。当屏幕捕获会话结束时，应用必须在 `MediaProjection.Callback` 的 `onStop()` 回调中显式调用 `VirtualDisplay.release()` 和 `MediaProjection.stop()` 来释放所有相关资源。否则，底层的图形缓冲区（Graphic Buffer）将无法被回收，导致内存被快速耗尽，这在应用堆栈分析中通常表现为大量未释放的 `byte` 数组。
- **硬件加速：**`VirtualDisplay` 的合成过程可以由硬件合成器（Hardware Composer, HWC）进行优化。HWC能够利用专门的硬件通路高效地合成图层，降低GPU负载。如果HWC不支持或未被使用，合成任务将回退到由OpenGL ES (GLES) 在GPU上执行，这会增加GPU的压力和功耗。在设计车载系统时，确保 `VirtualDisplay` 的合成路径得到硬件加速是至关重要的性能考量。

1.3 技术对比：为车载场景选择合适的工具

综上所述，`Presentation API` 和 `VirtualDisplay` 是为不同目的设计的工具，其在车载领域的适用性差异巨大。

API特性	Presentation API	VirtualDisplay API
核心概念	特化的 <code>Dialog</code> ，与父 <code>Activity</code> 绑定	将内容流式传输到应用提供的 <code>Surface</code>
主要用途	在副屏上显示简单的、非交互式的附属内容	屏幕捕获、录制、流式传输、复杂的离屏渲染
实现复杂度	低（API简单直观）	高（需处理 <code>MediaProjection</code> 授权、 <code>Surface</code> 管理、生命周期）
资源开销	较低	较高（涉及完整的屏幕内容合成管线）
交互能力	非常有限（无原生触摸事件处理）	高（取决于 <code>Surface</code> 的消费端，如可交互的 <code>SurfaceView</code> ）
音频焦点	与父应用共享（重大局限）	不直接管理（取决于消费 <code>surface</code> 的应用如何处理音频）
理想车载场景	仪表盘显示简单的导航转向箭头、媒体元数据	后排娱乐屏内容镜像、自定义仪表盘渲染、行车记录功能

从上述分析可以看出，Android原生的多屏显示API并非为现代汽车座舱中丰富的、多用户、并发娱乐的复杂场景而生。它们是通用的底层工具，暴露了系统的基本能力。`Presentation` 过于简单，无法满足独立娱乐的需求；而 `VirtualDisplay` 虽功能强大，但其本质是屏幕内容的“搬运工”，而非一个用于在副屏上独立运行和管理应用的框架。

这个原生框架在架构层面的空白，正是催生Android Automotive OS (AAOS) 复杂的多屏及多音区管理体系的根本原因。OEM厂商无法简单地“开箱即用”这些API来打造有竞争力的座舱体验，而必须依赖AAOS提供的高级服务，或投入巨大资源进行深度自研。

第二章：Android Automotive OS (AAOS) 的架构上下文

AAOS在标准Android框架之上，构建了一个专为车载环境设计的管理与策略层。它的核心目标是解决原生Android所忽略的特定挑战：多用户独立会话、音频隔离、行车安全以及OEM的深度定制需求。

2.1 在AAOS栈中集成显示器

AAOS的架构分为应用层、框架层（含Car API）、服务层（CarService）和车辆硬件抽象层（VHAL）。显示管理逻辑跨越了多个层级，其核心在于引入了“乘客域”的概念。

- **关键服务角色：**
 - **DisplayManagerService**：标准的Android服务，负责管理物理和虚拟显示器的生命周期。
 - **CarService**：AAOS的神经中枢，是所有车载服务的中心，负责与VHAL通信，获取车辆状态（如车速、档位）。
 - **CarOccupantZoneManager**：这是AAOS的关键创新。它将车内的物理座位（如驾驶位、副驾驶位、左后乘客位）映射为唯一的 `occupantZoneId`。当用户登录某个屏幕时，系统会将该用户的ID、其所在的 `occupantZoneId`、其正在使用的显示屏以及对应的音频区域（`audioZoneId`）进行绑定。这一机制使得系统能够精确地知道“谁”在“哪个位置”使用“哪个屏幕”，从而实现个性化和权限隔离。
- **显示器映射**：通过这种方式，AAOS超越了原生Android简单的“主屏/副屏”二元模型，建立了一个以乘客为中心的、更精细化的管理模型。OEM厂商通过XML配置文件（如 `car_audio_configuration.xml`）来定义 `occupantZoneId` 与 `audioZoneId` 的映射关系，为每个座位创建统一的体验档案。

2.2 克服核心挑战：多音区音频管理

AAOS多音区音频子系统是其架构设计的典范，它直接解决了 `Presentation API` 单一音频焦点的痛点。

- **动态音频区 (Dynamic Audio Zones)**：AAOS允许将车内划分为多个独立的音频区。每个音频区都拥有自己独立的音频焦点、音量控制和路由策略。这正是实现驾驶员听导航、乘客看电影且互不干扰的关键技术。
- **基于策略的路由**：系统通过 `car_audio_configuration.xml` 文件定义每个音频区的属性。OEM可以精细地将不同的音频用途（`AudioAttributes.USAGE`，如 `USAGE_MEDIA`、`USAGE_NAVIGATION`、`USAGE_ASSISTANCE`）路由到每个区域内不同的物理输出设备（总线端口）。例如，导航音频（`USAGE_NAVIGATION`）可以被配置为仅在主驾驶区的头枕扬声器播放，而媒体音频（`USAGE_MEDIA`）则在全车扬声器播放。
- **高级音频功能**：随着AAOS版本的迭代，还引入了更高级的功能，如动态区域配置（允许乘客在后排头枕扬声器和蓝牙耳机间切换）和音频镜像/投射（允许不同区域的乘客收听同一音频源）。

2.3 OEM级别的定制与控制

AAOS为汽车制造商提供了极高的定制自由度，以打造符合其品牌定位和安全要求的HMI。

- **配置系统装饰 (System Decorations)**：OEM可以为每个显示屏独立配置是否显示系统UI元素，如壁纸、导航栏、应用启动器等。这些配置定义在 `display_settings.xml` 中，由 `DisplayWindowSettings` 服务在系统启动时加载和应用。这对于确保驾驶员前方的屏幕（如仪表盘）界面简洁、无干扰至关重要。
- **安全与访问限制**：AAOS拥有强大的安全模型。任何显示屏都可以被标记为私有（`Display#FLAG_PRIVATE`），从而限制只有系统进程或创建该显示屏的应用（即“所有者”）才能在其上添加内容或启动 `Activity`。这一机制是防止恶意第三方应用创建虚拟显示屏来窃取用户敏感信息（如通过屏幕录制）或进行UI欺骗的关键保障。例如，系统严格限制了应用启动器（Launcher）只能在系统拥有的显示屏上运行。

AAOS的多屏架构本质上是在标准Android显示框架之上构建的一个强大的“管理与策略层”。它并没有重新发明轮子，而是通过引入 `OccupantZone`、`AudioZone` 等车载特定概念，并利用 `CarService` 作为策略执行中心，解决了标准Android在多用户并发、音频隔离、行车安全等方面的缺失。OEM选择AAOS，不仅是选择了一个操作系统，更是采纳了一套预置的、针对车载环境复杂性的成熟架构解决方案，这极大地节省了研发成本，但也意味着在架构上需要遵循谷歌的设计理念。

第三章：案例研究：理想L系列的“原生集成”路线

理想L系列车型（如L9）的智能座舱是车载多屏体验的标杆之一。它运行在深度定制的Android系统之上，通过强大的硬件和复杂的软件工程，实现了远超原生Android能力的多屏交互。其技术路线可以被归纳为“中心化增强”或“原生集成”模式。

3.1 系统架构：双骁龙8155驱动的五屏矩阵

- **硬件配置：**理想L9的座舱核心由两颗高通骁龙8155车载SoC驱动，共同支撑着一个复杂的五屏系统：HUD抬头显示、方向盘交互屏、中控与副驾双15.7英寸OLED联屏，以及后舱娱乐屏。对 Desay SV（德赛西威）供应的智能座舱域控制器的拆解分析也证实，其内部集成了强大的高通处理器，硬件成本高昂，性能强劲。
- **架构启示：**采用两颗顶级座舱SoC的配置，明确指向了一种“中心化计算”的架构。所有屏幕的图形渲染、应用运行和状态管理都由这个强大的中央处理单元负责。这与后续将要分析的鸿蒙分布式架构形成了鲜明对比。其核心挑战在于，如何让一个中央系统同时、高效地驱动多个高分辨率显示屏，并保证低延迟的交互响应。

3.2 解构副驾与后排娱乐屏的实现

`Presentation API` 因其在音频和交互上的根本性限制，显然无法满足理想L系列对副驾和后排乘客提供独立、完整娱乐体验的需求。其技术实现远比调用原生API复杂，很可能是一套基于AOSP（Android开源项目）的深度定制框架。

- **假设的技术实现路径：**
 1. **定制化的WindowManager与ActivityManager：**理想的工程师很可能修改了Android框架层的核心服务。通过对 `WindowManagerService` 和 `ActivityManagerService` 的改造，允许将完整的 `Activity` 实例指定到特定的副屏（非 `DEFAULT_DISPLAY`）上启动和运行，从而绕开了 `Presentation` 基于 `Dialog` 的限制。这使得副驾屏可以像一个独立的Android设备一样运行功能完整的应用程序。
 2. **用于交互的自定义系统服务：**为了实现如“三指滑动”这类在不同屏幕间流转内容的高级交互功能（此功能在问界M9上被提及，是高端多屏座舱的代表性功能），理想必须开发一个自定义的系统服务。该服务在底层监听特定的手势事件（如多点触控），捕获源应用的关键状态（如视频播放进度、浏览的页面URL、登录凭证等），然后通过IPC（进程间通信）通知 `ActivityManagerService` 在目标屏幕上重新启动或恢复该应用，并将捕获的状态传递过去。
 3. **VirtualDisplay的辅助应用：**虽然 `VirtualDisplay` 不太可能作为运行副驾屏主UI的机制，但它可能被用于一些特定的辅助功能。例如，实现后排乘客的手势控制，系统通过摄像头捕捉手部动作，然后将光标渲染在一个覆盖于屏幕上层的 `VirtualDisplay` 中，模拟鼠标操作。或者，在某些模式下，用于将一个屏幕的内容简单镜像到另一个屏幕。

3.3 工程挑战：状态同步、延迟与并发输入

在理想这种中心化的多屏架构下，软件工程团队面临着巨大的挑战：

- **状态同步 (State Synchronization)**：这是最核心的难题。当一个应用在屏幕间“流转”时，如何保证其状态的无缝迁移是决定用户体验好坏的关键。这不仅要求一个高效可靠的IPC机制，更需要应用本身进行适配，能够将其核心状态序列化、传递并恢复。对于第三方应用而言，这几乎是不可能完成的任务，因此这类高级交互通常仅限于系统预装应用。
- **性能与延迟**：同时驱动多块3K分辨率的OLED屏幕，对SoC的GPU渲染能力、内存带宽和系统总线都提出了极致要求。双8155芯片的配置正是为了应对这一挑战。系统必须确保所有屏幕的触摸延迟都足够低，并且在执行内容滑动等消耗大量资源的动画时，界面依然保持流畅。
- **并发输入与音频**：系统必须能同时处理来自驾驶员、副驾乘客的触摸输入，以及可能来自后排乘客的手势或遥控器输入。同时，必须实现严格的音频隔离，确保不同屏幕的音频流被正确路由到各自的音频区（如驾驶员头枕音响、副驾耳机插孔、后排扬声器）。这要求一套功能完备、高度定制的、类似AAOS的多音区音频架构。

理想L系列的座舱系统，代表了在Android原生框架上实现极致多屏体验的“强力整合”路线。它没有选择另起炉灶，而是利用顶级的硬件资源，并投入巨大的软件工程力量，对标准Android的核心组件进行深度修改和扩展，以弥补其原生能力的不足。这种方法在最大程度上保持了对Android应用生态的兼容性，但代价是极高的研发和维护成本。它展示了AOSP的灵活性，也反衬出要将其应用于高端车载座舱所需付出的巨大架构努力。

<https://www.lixiang.com/community/detail/article/75527.html>